

Phase 6 — Notebook 2: Drift Detection Report

This notebook detects and quantifies feature and prediction drift for both ML models using **Evidently AI**.

Approach:

- Use `model_table.csv` (25,000 rows) split time-based into **reference** (first 80%) and **current** (last 20%)
- Run `DataDriftPreset` + `TargetDriftPreset` for both model feature subsets
- Export a standalone HTML drift report
- Interpret findings and recommend monitoring thresholds

```
In [1]: from pathlib import Path

import pandas as pd

REPO_ROOT = Path(".").resolve().parent.parent
DB_PATH = REPO_ROOT / "Database" / "model_table.csv"
REPORT_DIR = Path(".").resolve()

df = pd.read_csv(DB_PATH, parse_dates=["visit_date", "billing_date", "registration_date"])
df = df.sort_values("visit_date").reset_index(drop=True)

print(f"Dataset shape: {df.shape}")
print(f>Date range: {df['visit_date'].min().date()} → {df['visit_date'].max().date()}")
```

Dataset shape: (25000, 33)
Date range: 2025-01-20 → 2026-01-20

1. Reference / Current Split

```
In [2]: split_idx = int(len(df) * 0.80)
reference = df.iloc[:split_idx].copy()
current = df.iloc[split_idx:].copy()

print(f"Reference set: {len(reference):,} rows  ({reference['visit_date'].min().date()} → {reference['visit_date'].max().date()})")
print(f"Current set:   {len(current):,} rows  ({current['visit_date'].min().date()} → {current['visit_date'].max().date()})")
```

Reference set: 20,000 rows (2025-01-20 → 2025-11-08)
Current set: 5,000 rows (2025-11-08 → 2026-01-20)

2. Feature Subsets

Drift is measured on the exact feature sets that each model was trained on.

```
In [3]: # Patient risk model features + target
RISK_COLS = ["chronic_flag", "gender", "visit_frequency", "risk_score"]

# Claim outcome model features + target
CLAIM_COLS = [
```

```

    "age", "gender", "city", "insurance_provider", "chronic_flag",
    "department", "visit_type", "length_of_stay_hours", "billed_amount",
    "payment_days", "visit_frequency", "lag_time", "claim_status"
]

# lag_time: billing_delay_days is the equivalent column in model_table
if "lag_time" not in df.columns and "billing_delay_days" in df.columns:
    df["lag_time"] = df["billing_delay_days"]
    reference["lag_time"] = reference["billing_delay_days"]
    current["lag_time"] = current["billing_delay_days"]

risk_ref = reference[RISK_COLS].copy()
risk_cur = current[RISK_COLS].copy()

claim_ref = reference[CLAIM_COLS].copy()
claim_cur = current[CLAIM_COLS].copy()

print("Risk subset – reference:", risk_ref.shape, " current:", risk_cur.shape)
print("Claim subset – reference:", claim_ref.shape, " current:", claim_cur.shape)

```

Risk subset – reference: (20000, 4) current: (5000, 4)
 Claim subset – reference: (20000, 13) current: (5000, 13)

3. Drift Detection — Patient Risk Model

```

In [4]: from evidently.legacy.pipeline.column_mapping import ColumnMapping
        from evidently.legacy.metric_preset import DataDriftPreset, TargetDriftPreset
        from evidently.legacy.report import Report

        risk_column_mapping = ColumnMapping(
            target="risk_score",
            prediction=None,
            numerical_features=["visit_frequency"],
            categorical_features=["chronic_flag", "gender"],
        )

        risk_report = Report(metrics=[DataDriftPreset(), TargetDriftPreset()])
        risk_report.run(reference_data=risk_ref, current_data=risk_cur, column_mapping=risk_column_mapping)

        print("Patient Risk drift report generated.")
        risk_report
    
```

Patient Risk drift report generated.



Dataset Drift

Dataset Drift is NOT detected. Dataset drift detection threshold is 0.5

4
Columns

0
Drifted Columns

0.0
Share of Drifted Columns

Data Drift Summary

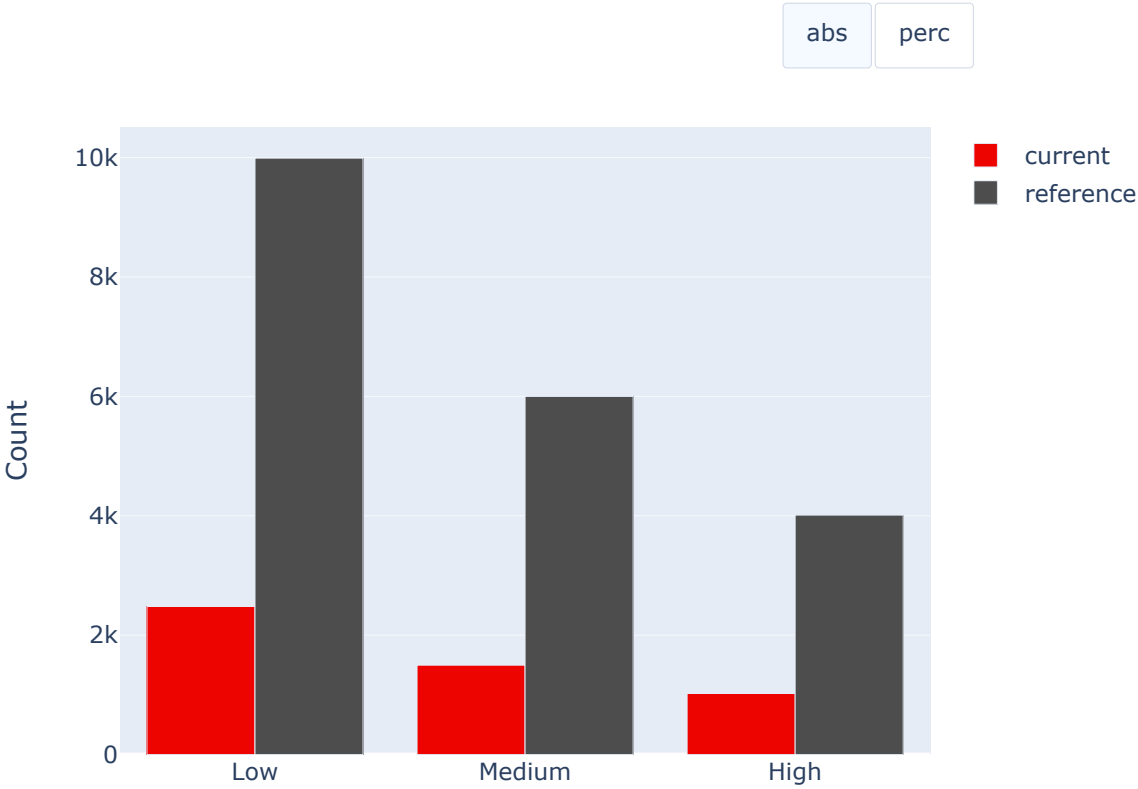
Drift is detected for 0.0% of columns (0 out of 4).

🔍 Search ✕						
Column	Type	Reference Distribution	Current Distribution	Data Drift	Stat Test	Drift Score
> risk_score	cat			Not Detected	Jensen-Shannon distance	0.00365
> visit_frequency	num			Not Detected	Wasserstein distance (normed)	0.00782
> chronic_flag	cat			Not Detected	Jensen-Shannon distance	0.006364
> gender	cat			Not Detected	Jensen-Shannon distance	0.00244

Drift in column 'risk_score'

Data drift not detected. Drift detection method: Jensen-Shannon distance. Drift score: 0.004

DATA DISTRIBUTION



4. Drift Detection — Claim Outcome Model

Correlations for column 'risk_score'.

```
In [5]: claim_column_mapping = ColumnMapping(
    target="claim_status",
    prediction=None,
    numerical_features=["age", "length_of_stay_hours", "billed_amount", "payment_days", "visit_frequency", "lag_time"],
    categorical_features=["gender", "city", "insurance_provider", "chronic_flag", "department", "visit_type"],
    cramer_v=
)

claim_report = Report(metrics=[DataDriftPreset(), TargetDriftPreset()])
claim_report.run(reference_data=claim_ref, current_data=claim_cur, column_mapping=claim_column_mapping)

print("Claim Outcome drift report generated.")
claim_report

Claim Outcome drift report generated.
```

Out[5]:



Dataset Drift

Dataset Drift is NOT detected. Dataset drift detection threshold is 0.5

5. Extract Drift Metrics Summary

```
In [ ]: from evidently.legacy.report import Report as _Report
def extract_drift_summary(report: _Report, model_name: str) -> pd.DataFrame:
    """Extract per-feature drift test results from an Evidently report."""
    results = report.as_dict()
    rows = []
    for metric in results.get("metrics", Columns):
        result = metric.get("result", {})
        drift_by_columns = result.get("drift_by_columns", {})
        for col_name, col_result in drift_by_columns.items():
            rows.append({
                "Model": model_name,
                "Feature": col_name,
                "Drift Detected": col_result.get("drift_detected", False),
                "Drift Score": round(col_result.get("drift_score", float("nan")), 4),
                "Test": col_result.get("statstest_name", ""),
                "Threshold": col_result.get("statstest_threshold", ""),
            })
    # deduplicate (DataDriftPreset and TargetDriftPreset may overlap)
    df_out = pd.DataFrame(rows).drop_duplicates(subset=["Model", "Feature"])
    return df_out

risk_summary = extract_drift_summary(risk_report, "patient_risk")
claim_summary = extract_drift_summary(claim_report, "claim_outcome")
```

```
combined = pd.concat([risk_summary, claim_summary], ignore_index=True)
display(combined)
```

```
Cell In[6], line 27
    combined = pd.concat([risk_summary, claim_summary], ignore_index=True)display(combined)
                                     ^
SyntaxError: invalid syntax
```

```
In [ ]: drift_count = combined[combined["Drift Detected"] == True]
print(f"Features with drift detected: {len(drift_count)} / {len(combined)}")
if len(drift_count):
    print("\nDrifted features:")
    display(drift_count[["Model", "Feature", "Drift Score", "Test"]])
```

6. Export HTML Drift Report

```
In [ ]: risk_html_path = REPORT_DIR / "patient_risk_drift_report.html"
claim_html_path = REPORT_DIR / "claim_outcome_drift_report.html"

risk_report.save_html(str(risk_html_path))
claim_report.save_html(str(claim_html_path))

print(f"Saved: {risk_html_path}")
print(f"Saved: {claim_html_path}")
```

7. Interpretation and Monitoring Thresholds

Drift Test Methods

Evidently automatically selects the test per feature type:

Feature Type		Default Test	Interpretation
Numerical		Wasserstein distance (or KS for small samples)	Score > 0.1 → moderate drift; > 0.2 → significant
Categorical		Chi-squared test	p-value < 0.05 → drift detected
Target (multiclass)		Chi-squared test	p-value < 0.05 → label distribution shifted

Recommended Retraining Triggers

Condition	Action
> 30% of features drift simultaneously	Immediate retraining investigation
Target label distribution drift detected	Priority retraining review
High-importance feature (billed_amount , visit_frequency) drifts	Retraining required
< 20% features drift, no target drift	Monitor — continue without retraining

Notes on This Simulation

The reference (80%) vs current (20%) split is time-based, using the same `model_table.csv` used for training. In production, the "current" data would be drawn from recent API call logs enriched with ground-truth labels (delayed by claim resolution cycle time, typically 30–90 days).